

Project Report: Smart Hotel Review Platform

Posem Reddy Susruth

Objective

To develop a smart and interactive hotel review platform where customers can submit and explore feedback, while hotel administrators gain actionable insights through real-time, AI-powered sentiment analysis. The system provides visual analytics, automated daily alerts with actionable suggestions, and a gamified user experience to support continuous service enhancement and customer satisfaction.

List of Modules

1. User Authentication
2. Review Submission & Viewer
3. AI-Powered Aspect-Based Sentiment Analyzer
4. Per-Review Aspect Sentiment Breakdown
5. Admin Dashboard with Analytics Visualizations
6. Daily Admin Alert & Actionable Suggestions
7. Heatmap Visualization by Aspect and Time
8. Automated Reporting

Detailed Description of the Modules

1. User Authentication

User login and role-based access are handled through **AWS Cognito**, managed by the **AWS Amplify** framework. This provides a secure, fully-managed authentication flow for both hotel guests (customers) and administrators.

2. Review Submission & Viewer

A responsive frontend (React.js) interacts with a secure, real-time **AWS API** managed by Amplify Data.

- Customers can submit detailed reviews.
- The API triggers a Lambda function (`submitReview`) for processing.
- Submitted reviews are stored in **Amazon DynamoDB** for fast, scalable retrieval.
- The frontend can query the AppSync API to display all reviews.

3. AI-Powered Aspect-Based Sentiment Analyzer

This is the core of the platform's intelligence.

- Uses the **Huggingface API** for powerful, real-time analysis.
- When a review is submitted, an AWS Lambda function (`submitReview`) securely calls the Huggingface API.
- The API extracts key aspects (e.g., "room", "service", "cleanliness") and their corresponding sentiments (positive, neutral, negative).
- This structured JSON data is then stored directly in the review item in Amazon DynamoDB.

4. Per-Review Aspect Sentiment Breakdown

Each review in the UI displays a clear breakdown of the AI-generated sentiment for each aspect (e.g., "Service:  Positive", "Cleanliness:  Negative"). This data is read directly from the `sentiments` map in the DynamoDB item, which was populated by the analyzer.

5. Admin Dashboard with Analytics Visualizations

A separate admin-only interface provides real-time visual dashboards using **Chart.js**. Graphs include bar charts, pie charts, and line graphs to visualize sentiment trends, aspect frequency, and overall satisfaction scores, all sourced directly from the DynamoDB database via the AppSync API.

6. Daily Admin Alert & Actionable Suggestions

A proactive alert system ensures administrators are always informed.

- An **Amazon EventBridge (Scheduler)** rule triggers a Lambda function (`dailyReviewAlert`) once every 24 hours.
- The Lambda function scans DynamoDB for all reviews submitted in the last 24 hours.
- It uses a local analysis function to aggregate sentiment scores and identifies aspects where negative/neutral feedback exceeds a 50% threshold.
- It formats a concise email (e.g., "Today there are 5 new reviews and aspects room, cleanliness need actions") and sends it to the admin via **Amazon SES (Simple Email Service)**.

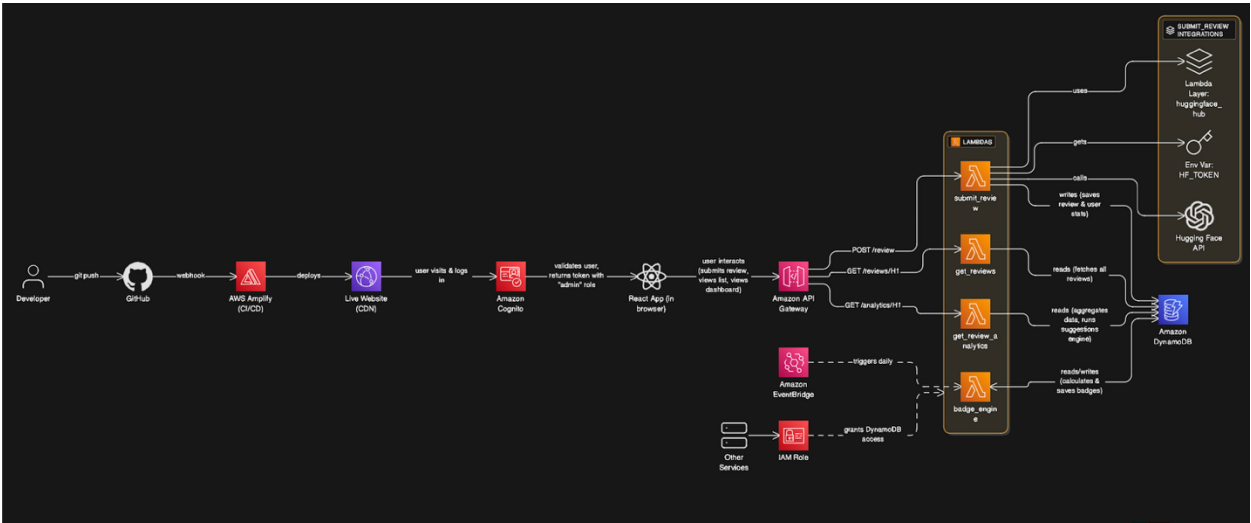
7. Gamified Feedback Tracker

Enhances user engagement by awarding badges like "Top Reviewer", "Early Bird", and "Most Helpful Review". All badge logic and tracking are implemented using a dedicated AWS Lambda function (`badgeEngine`) and stored in DynamoDB.

8. Automated Reporting

A module for generating and downloading reports. The `dailyReviewAlert` function serves as the basis for this, which can be expanded to generate weekly/monthly PDF or CSV reports that are stored in Amazon S3.

Architecture Contribution



Task	Work Done
Frontend UI	Designed customer and admin dashboards using React, Chart.js, and heatmap.js.
Authentication	Integrated AWS Cognito via the AWS Amplify framework for secure role-based login.
NLP	Integrated Google Gemini API for real-time aspect-based sentiment analysis in the backend.
Backend	Built serverless backend using AWS Amplify Gen 2 (TypeScript) .
Functions	Developed Lambda functions for review submission (<code>submitReview</code>), gamification (<code>badgeEngine</code>), and scheduled alerts (<code>dailyReviewAlert</code>).
API	Implemented a secure AWS API via Amplify Data.
Alerting	Built a complete admin alert system using Amazon EventBridge and Amazon SES .
Cloud	Integrated and deployed full stack using AWS Free Tier services, managed via Amplify.

Software & AWS Services

Software Requirements

Area	Tools
------	-------

Frontend HTML/CSS, React.js, Chart.js, heatmap.js
NLP Google Gemini API (called via `fetch` from AWS Lambda)
Backend AWS Amplify (Gen 2, TypeScript), AWS Lambda (Node.js)
Auth AWS Cognito
Database Amazon DynamoDB
Email Amazon SES
Scheduling Amazon EventBridge
Monitoring AWS CloudWatch

AWS Services Used

No.	Service	Purpose in Project
1	AWS Amplify	Core framework for managing the entire backend (Auth, API, Data, Functions).
2	Amazon Cognito	User authentication (sign-up, login, role-based access for customers & admins).
3	AWS Lambda	Serverless backend logic: <code>submitReview</code> (sentiment analysis), <code>dailyReviewAlert</code> (analysis & email).
4	Amazon DynamoDB	NoSQL database for storing reviews, aspect sentiment, and user data.
5	Amazon EventBridge	Serverless scheduler to trigger the <code>dailyReviewAlert</code> function every 24 hours.
6	Amazon SES	(Simple Email Service) Used by the alert function to send daily summary emails to the admin.
7	Amazon S3	Storage for Amplify deployment files and (future) generated reports.
8	AWS CloudWatch	Provides logs for Lambda and API activity, used for debugging.
10	AWS Secrets Manager	Used to store the API key for the huggingface model used
11	AWS IAM	Manages role-based permissions for all AWS services to communicate securely.
12	AWS API Gateway	Used to connect all the services

References

1. AWS Free Tier Documentation
<https://aws.amazon.com/free/>
2. Amazon Cognito Developer Guide

<https://docs.aws.amazon.com/cognito/latest/developerguide/what-is-amazon-cognito.html>

3. **Amazon DynamoDB Developer Guide**

<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Introduction.html>

4. **AWS Lambda Developer Guide**

<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>

5. **Amazon S3 Documentation**

<https://docs.aws.amazon.com/s3/index.html>

6. **Amazon API Gateway Documentation**

<https://docs.aws.amazon.com/apigateway/>

7. **Hugging Face Transformers**

<https://huggingface.co/docs/transformers/index>

8. **Chart.js Documentation**

<https://www.chartjs.org/docs/latest/>

9. **Heatmap.js Library**

<https://www.patrick-wied.at/static/heatmapjs/>

10. **WeasyPrint PDF Generator**

<https://weasyprint.readthedocs.io/en/stable/>

11. **AWS Lambda Layers Documentation**

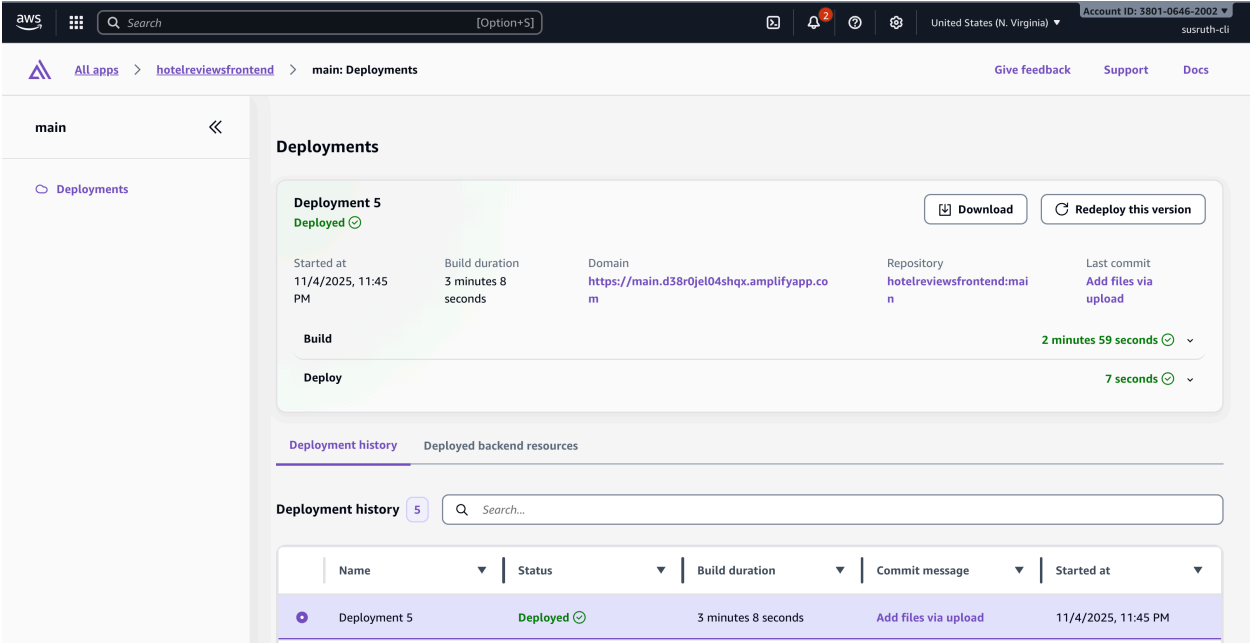
<https://docs.aws.amazon.com/lambda/latest/dg/configuration-layers.html>

12. **Flask Web Framework (for optional PDF generation backend)**

<https://flask.palletsprojects.com/>

Screenshots

AWS Amplify



Amazon Cognito

Amazon Cognito > User pools > hotelreviewsfrontend01d3a8ce_userpool_01d3a8ce-dev > Overview

Amazon Cognito < Current user pool View all hotelreviewsfrontend01d3a8ce-dev Overview Applications App clients User management Users Groups

Overview: hotelreviewsfrontend01d3a8ce_userpool_01d3a8ce-dev Info Rename Delete user pool

User pool information

User pool name

hotelreviewsfrontend01d3a8ce_userpool_01d3a8ce-dev

Token signing key URL

https://cognito-idp.us-east-1.amazonaws.com/us-east-1_JU4FzBpgd/.well-known/jwks.json

User pool ID

us-east-1_JU4FzBpgd

Estimated number of users

4

ARN

arn:aws:cognito-idp:us-east-1:380106462002:userpool/us-east-1_JU4FzBpgd

Feature plan

Essentials

Created time

September 20, 2025 at 16:02 GMT+5:30

Last updated time

September 20, 2025 at 16:02 GMT+5:30

Recommendations

Amazon Cognito > User pools > hotelreviewsfrontend01d3a8ce_userpool_01d3a8ce-dev > Users

Amazon Cognito < Current user pool View all hotelreviewsfrontend01d3a8ce-dev Overview Applications App clients User management Users Groups Authentication Authentication methods Sign-in Sign-up Social and external providers Extensions

Users Info Delete user Create user

Users (4) Info View, edit, and create users in your user pool. Users that are enabled and confirmed can sign in to your user pool.

Property: User name Search users by attribute

	User name	Email address	Email verified	Confirmation status	Status
☐	23bce5060	susruthbunny@gmail.com	Yes	Confirmed	Enabled
☐	tunir	2005really2005@gmail.com	Yes	Confirmed	Enabled
☐	sruthi	posemreddy.susruth2023...	Yes	Confirmed	Enabled
☐	admin	susruthreddy2811@gmail...	Yes	Confirmed	Enabled

Import users (0) Info Start Stop Create import job

View and create user CSV import jobs. Amazon Cognito can import users into this user pool from a specially-formatted CSV file. You can't import user passwords.

Search import jobs by job name

Amazon Cognito > User pools > hotelreviewsfrontend01d3a8ce_userpool_01d3a8ce-dev > Groups

Amazon Cognito < Current user pool View all hotelreviewsfrontend01d3a8ce-dev Overview Applications App clients User management Users Groups Authentication Authentication methods Sign-in Sign-up Social and external providers Extensions

Groups Info Delete Create group

Groups (1) Info Configure groups and add users. Groups can be used to add permissions to the access token for multiple users.

Filter groups by name and description

	Group name	Description	Precedence	Created time
☐	admins	-	0	2 months ago

Set up group-based access control with Amazon Verified Permissions With Verified Permissions, you can create policies that authorize access to API Gateway with user pool groups. Learn more

Go to Amazon Verified Permissions

AWS Lambda

AWS Lambda Functions console screenshot showing a list of 6 functions. The functions are:

Function name	Description	Package type	Runtime	Last modified
get_reviews	-	Zip	Python 3.13	1 week ago
badge_engine	-	Zip	Python 3.9	2 months ago
generate_review_report	-	Zip	Python 3.12	1 week ago
dailyReviewAnalyzer	-	Zip	Node.js 22.x	1 week ago
submit_review	-	Zip	Python 3.12	1 week ago
amplify-hotelreviewsfront-UpdateRolesWithIDPFuncti-r2Zq9XdspyeP	-	Zip	Node.js 22.x	2 months ago

get_reviews

```
import json

import boto3
from boto3.dynamodb.conditions import Key
from decimal import Decimal

class DecimalEncoder(json.JSONEncoder):
    def default(self, o):
        if isinstance(o, Decimal):
            return float(o)
        return super(DecimalEncoder, self).default(o)

db = boto3.resource('dynamodb')
table = db.Table('HotelReviews')

def lambda_handler(event, context):
    headers = {
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Headers': 'Content-Type',
        'Access-Control-Allow-Methods': 'GET, OPTIONS'
    }

    try:
        path = event.get('rawPath', '')
        hotel_id = path.split('/')[1]

        if not hotel_id:
            raise ValueError("hotelId is empty after parsing path.")

        response = table.query(
```

```

        IndexName='GSI1',
        KeyConditionExpression=Key('GSI1PK').eq(f"HOTEL#{hotel_id}"),
        ScanIndexForward=False
    )

    reviews = []
    for item in response.get('Items', []):
        reviews.append({
            'reviewId': item['PK'].split('#')[-1],
            'text': item.get('text'),
            'rating': item.get('rating'),
            'badges': item.get('badges', []) # <-- NEW: Get the badges
        })

    return {
        'statusCode': 200,
        'headers': headers,
        'body': json.dumps(reviews, cls=DecimalEncoder)
    }

except Exception as e:
    print(f"Error: {e}")
    return {
        'statusCode': 500,
        'headers': headers,
        'body': json.dumps({'error': str(e)})
    }

```

badge_engine

```

import json
import boto3
from collections import defaultdict
db = boto3.resource('dynamodb')
table = db.Table('HotelReviews')

def handler(event, context):
    print("Starting badge engine...")

    user_review_counts = defaultdict(int)

    paginator = db.meta.client.get_paginator('scan')
    pages = paginator.paginate(Table_name='HotelReviews')
    for page in pages:
        for item in page['Items']:

```

```

        if item.get('PK', '').startswith('REVIEW#') and 'userId' in item:
            user_id = item['userId']

            if user_id != 'anonymous':
                user_review_counts[user_id] += 1

    print(f"Found reviews for {len(user_review_counts)} users.")
    for user_id, count in user_review_counts.items():
        badges = []
        if count >= 1:
            badges.append('First Review')
        if count >= 5:
            badges.append('Enthusiast')
        if count >= 10:
            badges.append('Top Contributor')

        if badges:
            print(f"Awarding badges {badges} to user {user_id}")

    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': 'Badge engine run completed successfully.',
            'processed_users': len(user_review_counts)
        })
    }
}

```

generate_review_report

```

import json
import boto3
from boto3.dynamodb.conditions import Key
from decimal import Decimal

class DecimalEncoder(json.JSONEncoder):
    def default(self, o):
        if isinstance(o, Decimal):
            return float(o)
        return super(DecimalEncoder, self).default(o)

db = boto3.resource('dynamodb')
table = db.Table('HotelReviews')

def lambda_handler(event, context):

```

```

headers = {
    'Access-Control-Allow-Origin': '*',
    'Access-Control-Allow-Headers': 'Content-Type',
    'Access-Control-Allow-Methods': 'GET, OPTIONS'
}

try:
    path = event.get('rawPath', '')
    hotel_id = path.split('/')[1]

    if not hotel_id:
        raise ValueError("hotelId is empty after parsing path.")

    response = table.query(
        IndexName='GSI1',
        KeyConditionExpression=Key('GSI1PK').eq(f"HOTEL#{hotel_id}")
    )

    items = response.get('Items', [])

    aspect_sentiments = {}
    for item in items:
        for aspect, scores in item.get('aspectSentiment', {}).items():
            if aspect not in aspect_sentiments:
                aspect_sentiments[aspect] = {'positive': [], 'negative': [],
'neutral': []}

                aspect_sentiments[aspect]['positive'].append(scores.get('Positive',
0))
                aspect_sentiments[aspect]['negative'].append(scores.get('Negative',
0))
                aspect_sentiments[aspect]['neutral'].append(scores.get('Neutral', 0))

    aspect_analytics = {}
    for aspect, scores in aspect_sentiments.items():
        count = len(scores['positive'])
        if count > 0:
            aspect_analytics[aspect] = {
                'avg_positive': sum(scores['positive']) / count,
                'avg_negative': sum(scores['negative']) / count,
                'avg_neutral': sum(scores['neutral']) / count
            }
        else:
            aspect_analytics[aspect] = { 'avg_positive': 0, 'avg_negative': 0,
'avg_neutral': 0 }

    suggestions = []

```

```

        if not items:
            suggestions.append("No review data yet. Submit some reviews to see suggestions.")
        else:
            for aspect, data in aspect_analytics.items():

                if data['avg_negative'] > 0.4: # If avg negative sentiment is over 40%
                    suggestions.append(f"🔴 {aspect.capitalize()} ratings are very low. This area needs immediate attention.")
                elif data['avg_positive'] < 0.5: # If avg positive sentiment is under 50%
                    suggestions.append(f"🟡 {aspect.capitalize()} ratings are mediocre. Look for ways to improve this experience.")

            if not suggestions and items:
                suggestions.append("✅ All sentiment scores are looking good! Keep up the great work.")

    final_response = {
        "chartData": aspect_analytics,
        "suggestions": suggestions
    }

    return {
        'statusCode': 200,
        'headers': headers,
        'body': json.dumps(final_response, cls=DecimalEncoder)
    }
except Exception as e:
    print(f"Error: {e}")
    return {
        'statusCode': 500,
        'headers': headers,
        'body': json.dumps({'error': str(e)})
    }
}

```

dailyReviewAnalyzer

```

import { DynamoDBClient } from '@aws-sdk/client-dynamodb';
import {
    DynamoDBDocumentClient,
    ScanCommand,
} from '@aws-sdk/lib-dynamodb';
import { SESv2Client, SendEmailCommand } from '@aws-sdk/client-sesv2';
const ddbClient = new DynamoDBClient({});

```

```

const ddbDocClient = DynamoDBDocumentClient.from(ddbClient);
const sesClient = new SESv2Client({});

// Environment Variables
const REVIEW_TABLE_NAME = process.env.REVIEW_TABLE_NAME;
const ADMIN_EMAIL = process.env.ADMIN_EMAIL;
const SENDER_EMAIL = process.env.SENDER_EMAIL;
function analyzeReviews(reviews) {
  const aspectCounts = new Map();

  for (const review of reviews) {
    if (!review.sentiments || typeof review.sentiments !== 'object') {
      continue;
    }

    for (const [aspect, sentiment] of Object.entries(review.sentiments)) {
      if (!aspectCounts.has(aspect)) {
        aspectCounts.set(aspect, { positive: 0, neutral: 0, negative: 0, total: 0 });
      }

      const counts = aspectCounts.get(aspect);

      if (sentiment === 'positive') counts.positive += 1;
      else if (sentiment === 'neutral') counts.neutral += 1;
      else if (sentiment === 'negative') counts.negative += 1;

      counts.total += 1;
    }
  }

  const aspectsToReview = [];
  for (const [aspect, counts] of aspectCounts.entries()) {
    if (counts.total === 0) continue;
    const problematicCount = counts.neutral + counts.negative;
    const problematicPercentage = (problematicCount / counts.total);

    if (problematicPercentage > 0.5) {
      aspectsToReview.push(aspect);
    }
  }
  return aspectsToReview;
}

export const handler = async (event) => {
  console.log('Daily review alert function triggered.');
```

```

  if (!REVIEW_TABLE_NAME || !ADMIN_EMAIL || !SENDER_EMAIL) {
    console.error('Missing environment variables!');
    return;
  }

```

```

}

// 1. Get timestamp for 24 hours ago
const twentyFourHoursAgo = new Date(
  Date.now() - 24 * 60 * 60 * 1000
).toISOString();

// 2. Scan DynamoDB for new reviews
const scanCommand = new ScanCommand({
  TableName: REVIEW_TABLE_NAME,
  FilterExpression: 'createdAt > :date',
  ExpressionAttributeValues: { ':date': twentyFourHoursAgo },
  ProjectionExpression: 'sentiments',
});

let allNewReviews = [];
try {
  const result = await ddbDocClient.send(scanCommand);
  allNewReviews = result.Items || [];
  console.log(`Found ${allNewReviews.length} new reviews.`);
} catch (error) {
  console.error('Error scanning DynamoDB:', error);
  return;
}

const newReviewCount = allNewReviews.length;

// 3. Analyze the reviews
const aspectsToReview = analyzeReviews(allNewReviews);

// 4. Build the email message
let aspectMessage = 'no aspects need actions';
if (aspectsToReview.length > 0) {
  aspectMessage = `aspects ${aspectsToReview.join(', ')} need actions`;
}

const finalMessage = `Today there are ${newReviewCount} new reviews and
${aspectMessage}.`;
const emailSubject = `Daily Review Report: ${newReviewCount} New Reviews`;

// 5. Send the email using SES
const emailBodyHtml = `
  <html>
    <head><style>body { font-family: Arial, sans-serif; font-size: 16px;
}</style></head>
    <body>
      <p>Hello Admin,</p>
      <p style="font-size: 18px; font-weight: bold;">${finalMessage}</p>

```

```

        </body>
    </html>
`;

const sendEmailCommand = new SendEmailCommand({
    FromEmailAddress: SENDER_EMAIL,
    Destination: { ToAddresses: [ADMIN_EMAIL] },
    Content: {
        Simple: {
            Subject: { Data: emailSubject, Charset: 'UTF-8' },
            Body: {
                Html: { Data: emailBodyHtml, Charset: 'UTF-8' },
                Text: { Data: `Hello Admin, ${finalMessage}`, Charset: 'UTF-8' },
            },
        },
    },
});

try {
    await sesClient.send(sendEmailCommand);
    console.log('Email sent successfully.');
```

submit_review

```

import json
import os
import uuid
from datetime import datetime
import boto3
from decimal import Decimal

# Try importing huggingface_hub from the layer
try:
    from huggingface_hub import InferenceClient
    hf_client = InferenceClient(token=os.getenv("HF_TOKEN"))
except Exception as e:
    print(f"Failed to load Hugging Face client: {e}")
    hf_client = None

# Initialize AWS services
```

```

db = boto3.resource('dynamodb')
table = db.Table('HotelReviews')

# Aspect mapping remains the same
ASPECT_KEYWORDS = {
    'room': ['room', 'bed', 'space', 'suite', 'ac', 'air conditioning'],
    'cleanliness': ['clean', 'dirty', 'hygiene', 'housekeeping', 'towels', 'linen'],
    'food': ['food', 'breakfast', 'dinner', 'restaurant', 'meal', 'menu'],
    'service': ['service', 'staff', 'reception', 'check-in', 'waiter', 'attendant'],
    'price': ['price', 'cost', 'expensive', 'cheap', 'value']
}

def map_aspects(text):
    text_low = text.lower()
    found = {}
    for aspect, kws in ASPECT_KEYWORDS.items():
        for kw in kws:
            if kw in text_low:
                found.setdefault(aspect, []).append(kw)
    return list(found.keys())

def detect_sentiment(text):
    if not hf_client:
        raise Exception("Hugging Face client is not available. Check layer or HF_TOKEN.")

    result = hf_client.text_classification(text)

    best_sentiment = max(result, key=lambda x: x['score'])
    label = best_sentiment['label']
    score = best_sentiment['score']

    overall_sentiment = label.upper()
    overall_scores = {'Positive': Decimal(0), 'Negative': Decimal(0), 'Neutral': Decimal(0)}

    if overall_sentiment == 'POSITIVE':
        overall_scores['Positive'] = Decimal(str(score))
    elif overall_sentiment == 'NEGATIVE':
        overall_scores['Negative'] = Decimal(str(score))

    return overall_sentiment, overall_scores

def lambda_handler(event, context):
    try:
        body = event.get('body')
        if isinstance(body, str):
            body = json.loads(body)

```

```

hotelId = body.get('hotelId')
text = body.get('text','')
rating = body.get('rating')

userId = "user-" + str(uuid.uuid4())
reviewId = str(uuid.uuid4())
createdAt = datetime.utcnow().isoformat()

aspects = map_aspects(text)
overall_sentiment, overall_scores = detect_sentiment(text)

aspectSentiment = {}
for aspect in aspects:
    aspectSentiment[aspect] = overall_scores

item = {
    'PK': f"REVIEW#{reviewId}",
    'SK': f"METADATA#{createdAt}",
    'GSI1PK': f"HOTEL#{hotelId}",
    'GSI1SK': createdAt,
    'hotelId': hotelId,
    'userId': userId,
    'text': text,
    'rating': int(rating),
    'createdAt': createdAt,
    'aspectSentiment': aspectSentiment,
    'overallSentiment': overall_sentiment
}

table.put_item(Item=item)

return {
    'statusCode': 200,
    'headers': {
        'Access-Control-Allow-Origin': '*',
        'Access-Control-Allow-Headers': 'Content-Type',
        'Access-Control-Allow-Methods': 'POST, OPTIONS'
    },
    'body': json.dumps({
        'message': 'Review stored successfully (with Hugging Face sentiment)',
        'reviewId': reviewId
    })
}

except Exception as e:
    print(f"An error occurred: {e}")
    return {'statusCode': 500, 'body': json.dumps({'error': str(e)})}

```

DynamoDB

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations

Reserved capacity

Settings

Search [Option+S]

United States (N. Virginia)

susruth-cll

Account ID: 3801-0646-2002

susruth-cll

Tables (1) Info

November 13, 2025, 20:35 (UTC+5:30)

Last updated

Actions

Delete

Create table

Filter by tag Any tag key

Filter by tag value Any tag value

Find tables

Name

Status

Partition key

Sort key

Indexes

Replication Regions

Deletion protection

Favorite

Read cap.

HotelReviews

Active

PK (S)

SK (S)

1

0

Off

Provisioned

DynamoDB

Dashboard

Tables

Explore items

PartiQL editor

Backups

Exports to S3

Imports from S3

Integrations

Reserved capacity

Settings

Search [Option+S]

United States (N. Virginia)

susruth-cll

Account ID: 3801-0646-2002

susruth-cll

HotelReviews

November 13, 2025, 20:35 (UTC+5:30)

Last updated

Actions

Explore table items

Settings

Indexes

Monitor

Global tables

Backups

Exports and streams

Per

Protect your DynamoDB table from accidental writes and deletes

Edit PITR

When you turn on point-in-time recovery (PITR), DynamoDB backs up your table data automatically so that you can restore to any given second in the preceding 1 to 35 days. Additional charges apply. [Learn more](#)

General information Info

Get live item count

Partition key PK (String)

Sort key SK (String)

Capacity mode Provisioned

Table status Active

Alerts No active alarms

Point-in-time recovery (PITR) Info

Item count 17

Table size 6.7 kilobytes

Average item size 395.29 bytes

Resource-based policy Info

Not active

Amazon Resource Name (ARN)

arn:aws:dynamodb:us-east-1:380106462002:table/HotelReviews

Additional info

aws

Search

[Option+S]

2

United States (N. Virginia)

Account ID: 3801-0646-2002

usruth-cli

Amazon EventBridge

Schedules

Developer resources

Sandbox

Quick starts

Buses

Event buses

Rules

Global endpoints

Archives

Replays

Pipes

Pipes

Scheduler

Schedules

Schedule groups

Integration

Partner event sources

API destinations

Connections Updated

Schema registry

Schemas

Documentation

How it works



Amazon EventBridge Scheduler

Amazon EventBridge Scheduler is a serverless scheduler that allows you to create, run, and manage tasks from one central, managed service. Highly scalable, EventBridge Scheduler allows you to schedule millions of tasks that can invoke any AWS service as a target.



Templated targets

Amazon EventBridge Scheduler supports templated targets that you can configure quickly to perform common API operations using Amazon SQS, Amazon SNS, Lambda, and EventBridge.



Universal targets

EventBridge Scheduler provides a universal target parameter that you can use to create customized triggers that targets more than 270 AWS services and over 6,000 API operations on a schedule.



Flexible time windows

Amazon EventBridge Scheduler supports flexible time windows, allowing you to disperse your schedules and improve the reliability of your triggers for use cases that do not require precise scheduled invocation of targets.



Retries

Amazon EventBridge Scheduler supports at-least-once event delivery to targets, ensuring that at least one delivery succeeds with a response from the target.

Schedules (1)

Disable Edit Delete Create schedule

Search loaded schedules

All states

All groups

< 1 >

☐

Schedule name

Schedule group

Status

Target

Target type

Last modified

☐

RunDailyReviewAlert

default

Enabled

dailyReviewAnalyzer

LAMBDA_Invoke

Nov 05, 2025, 14:54:04 (UTC+00:00)

aws

Search

[Option+S]

2

United States (N. Virginia)

Account ID: 3801-0646-2002

usruth-cli

Amazon EventBridge

Schedules

RunDailyReviewAlert

Developer resources

Sandbox

Quick starts

Buses

Event buses

Rules

Global endpoints

Archives

Replays

Pipes

Pipes

Scheduler

Schedules

Schedule groups

Integration

Partner event sources

API destinations

Connections Updated

Schema registry

Schemas

RunDailyReviewAlert

Disable Edit Delete

Schedule detail

Schedule name

RunDailyReviewAlert

Description

-

Schedule group name

default

Status

Enabled

Schedule ARN

arn:aws:scheduler:us-east-1:380106462002:schedule/default/RunDailyReviewAlert

Action after completion

NONE

Schedule start time

-

Schedule end time

-

Execution time zone

Asia/Calcutta

Flexible time window

5 minutes

Created date

Nov 05, 2025, 20:24:04 (UTC+05:30)

Last modified date

Nov 05, 2025, 20:24:04 (UTC+05:30)

Schedule

Target

Retry policy

Dead-letter queue

Encryption

Schedule

Cron expression Info

30

23

*

*

?

*

Minutes Hours Day of month Month Day of week Year

Copy cron expression

Amazon SES

Amazon SES

Get set up

Account dashboard

Reputation metrics

SMTP settings

What's new

▼ **Configuration**

Identities

Configuration sets

Tenants [New](#)

[Dedicated IPs](#)

Global endpoints

Email templates

Suppression list

Email receiving

▼ **WorkMail**

Overview

Get set up [L](#)

▼ **Virtual Deliverability Manager**

Overview

▼ **Mail Manager**

Overview

Get set up

Identities (2) [Info](#)

Last updated Just now [Check for recommendations](#) [Send test email](#) [Delete](#) [Create identity](#)

☐	Identity	Identity type	Identity status
☐	susruthbunny@gmail.com	Email address	Verified
☐	susruthreddy2811@gmail.com	Email address	Verified

Recommendations (0) [Info](#)

To see high-impact recommendations, select up to 10 identities in the list above and choose Check for recommendations. If you implement a provided solution, refresh the table by selecting Check for recommendations to ensure it's no longer listed. If you enable Virtual Deliverability Manager, this table will refresh automatically for all your identities.

Impact	Identity name	Age	Recommendation/Description	Last checked	Resolve issue
No recommendations found					

Select up to 10 verified domain identities and select **Check for recommendations** on the Identities table above.

Amazon S3

Amazon S3

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access Grants

Access Points (General Purpose Buckets, FSx file systems)

Access Points (Directory Buckets)

Object Lambda Access Points

Multi-Region Access Points

Batch Operations

IAM Access Analyzer for S3

Block Public Access settings for this account

▼ **Storage Lens**

Dashboards

Storage Lens groups

AWS Organizations settings

Feature spotlight **11**

General purpose buckets (2) [Info](#)

[Copy ARN](#) [Empty](#) [Delete](#) [Create bucket](#)

Buckets are containers for data stored in S3.

☐	Name	AWS Region	Creation date
☐	amplify-hotelreviewsfrontend-dev-6ed14-deployment	US East (N. Virginia) us-east-1	September 20, 2025, 15:57:04 (UTC+05:30)
☐	hotel-reports-susruth-12345	US East (N. Virginia) us-east-1	September 20, 2025, 15:50:42 (UTC+05:30)

Account snapshot [Info](#)

Updated daily

[View dashboard](#)

Storage Lens provides visibility into storage usage and activity trends.

External access summary - new

Updated daily

[Info](#)

External access findings help you identify bucket permissions that allow public access or access from other AWS accounts.

These in the below picture are used to add these as layers in aws lambda functions

Amazon S3

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access Grants

Access Points (General Purpose Buckets, FSx file systems)

Access Points (Directory Buckets)

Object Lambda Access Points

Multi-Region Access Points

Batch Operations

IAM Access Analyzer for S3

Block Public Access settings for this account

Storage Lens

Dashboards

Storage Lens groups

AWS Organizations settings

Feature spotlight 11

AWS Marketplace for S3

hotel-reports-susruth-12345

Objects (2)

Copy S3 URICopy URLOpenDeleteActionsCreate folderUpload

Find objects by prefix

Name	Type	Last modified	Size	Storage class
huggingface_layer.zip	zip	September 23, 2025, 15:23:40 (UTC+05:30)	6.4 MB	Standard
reportlab_layer.zip	zip	September 22, 2025, 00:09:16 (UTC+05:30)	8.8 MB	Standard

Amazon S3

General purpose buckets

Directory buckets

Table buckets

Vector buckets

Access Grants

Access Points (General Purpose Buckets, FSx file systems)

Access Points (Directory Buckets)

Object Lambda Access Points

Multi-Region Access Points

Batch Operations

IAM Access Analyzer for S3

Block Public Access settings for this account

Storage Lens

Dashboards

Storage Lens groups

AWS Organizations settings

Feature spotlight 11

amplify-hotelreviewsfrontend-dev-6ed14-deployment

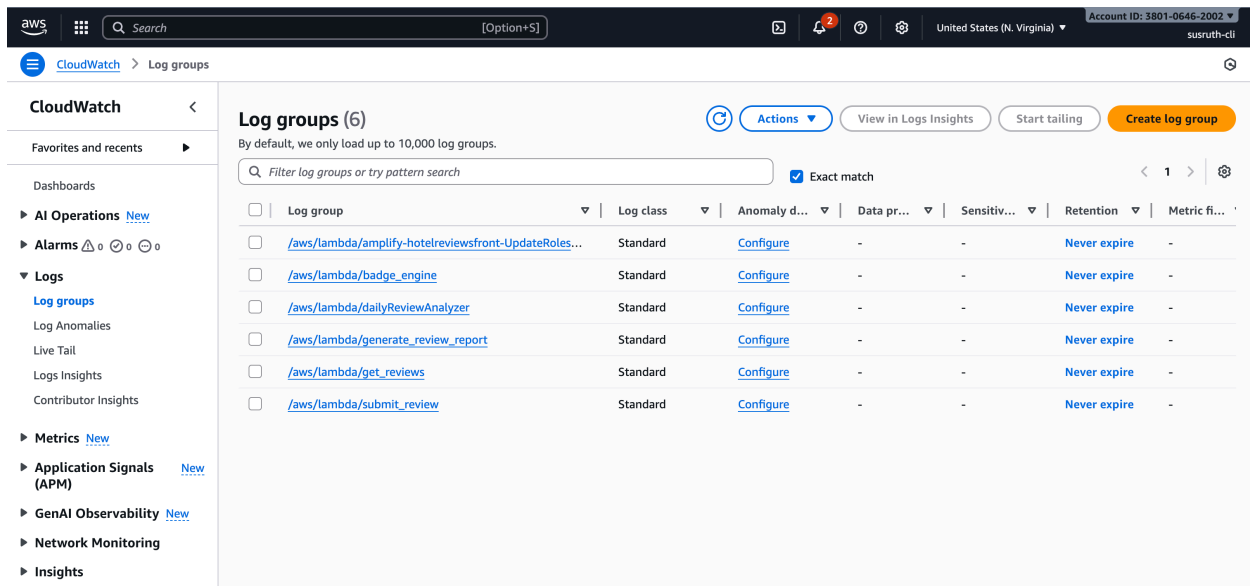
Objects (6)

Copy S3 URICopy URLOpenDeleteActionsCreate folderUpload

Find objects by prefix

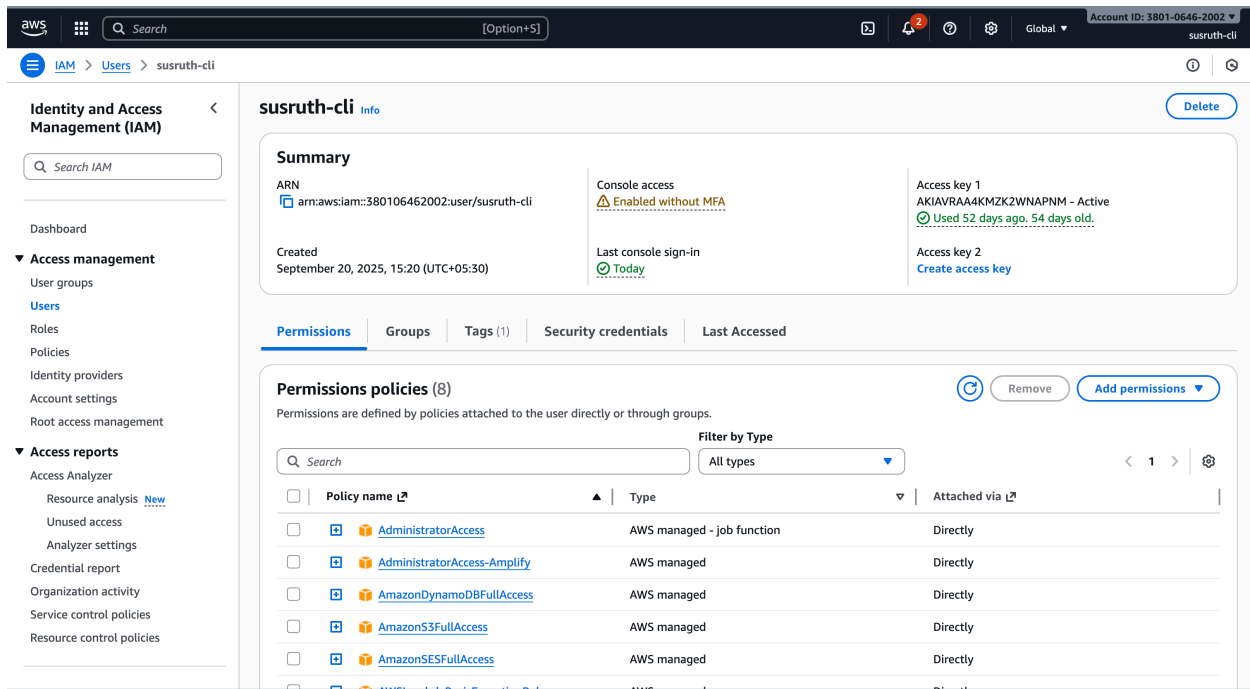
Name	Type	Last modified	Size	Storage class
#current-cloud-backend.zip	zip	November 4, 2025, 23:47:28 (UTC+05:30)	16.4 KB	Standard
amplify-cfn-templates/	Folder	-	-	-
amplify-meta.json	json	November 4, 2025, 23:47:29 (UTC+05:30)	2.3 KB	Standard
backend-config.json	json	November 4, 2025, 23:47:29 (UTC+05:30)	628.0 B	Standard
root-cloudformation-stack.json	json	November 4, 2025, 23:46:58 (UTC+05:30)	15.1 KB	Standard
studio-backend/	Folder	-	-	-

AWS CloudWatch



AWS IAM

the below user is solely created to make this project with the bare minimum access



The below shown roles is used to make all the services

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

Users

Roles

Policies

Identity providers

Account settings

Root access management

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Service control policies

Resource control policies

Roles (12)

Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Search

☐	Role name	Trusted entities	Last activity
☐	Amazon_EventBridge_Scheduler_LAMBDA_06f79959b4	AWS Service: scheduler	21 hours ago
☐	amplify-hotelreviewsfrontend-dev-6ed14-authRole	Identity Provider: cognito-identity.ai	-
☐	amplify-hotelreviewsfrontend-dev-6ed14-authRole-idp	AWS Service: lambda	54 days ago
☐	amplify-hotelreviewsfrontend-dev-6ed14-unauthRole	Identity Provider: cognito-identity.ai	-
☐	amplifyconsole-backend-role	AWS Service: amplify	8 days ago
☐	AWSServiceRoleForAPIGateway	AWS Service: ops.apigateway (Service-Linker)	-
☐	AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Service-Linker)	29 minutes ago
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Linker)	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service-Linker)	-
☐	CloudTrail_CloudWatchLogs_Role	AWS Service: cloudtrail	63 days ago
☐	lambda-role	AWS Service: lambda	13 hours ago
☐	upClientLambdaRole01d3a8ce6ed14-dev	AWS Service: lambda	-

Roles Anywhere

Info

Authenticate your non AWS workloads and securely provide access to AWS services.

Manage

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Roles

lambda-role

Identity and Access Management (IAM)

Search IAM

Dashboard

Access management

Users

Roles

Policies

Identity providers

Account settings

Root access management

Access reports

Access Analyzer

Resource analysis

Unused access

Analyzer settings

Credential report

Organization activity

Service control policies

Resource control policies

lambda-role

Info

Allows Lambda functions to call AWS services on your behalf.

Delete

Summary

Edit

Creation date

September 20, 2025, 15:38 (UTC+05:30)

Last activity

13 hours ago

ARN

arn:aws:iam::380106462002:role/lambda-role

Maximum session duration

1 hour

Permissions

Trust relationships

Tags

Last Accessed

Revoke sessions

Permissions policies (8)

Info

You can attach up to 10 managed policies.

Simulate

Remove

Add permissions

Search

Filter by Type

All types

☐	Policy name	Type	Attached entities
☐	AmazonDynamoDBFullAccess	AWS managed	2
☐	AmazonEventBridgeFullAccess	AWS managed	1
☐	AmazonS3FullAccess	AWS managed	2
☐	AmazonSESFullAccess	AWS managed	2
☐	AWSLambdaBasicExecutionRole	AWS managed	2

AWS Secrets Manager

aws

Search

[Option+S]

2

United States (N. Virginia)

Account ID: 3801-0646-2002

susruth-cli

AWS Secrets Manager

Secrets

HuggingFaceAPIToken

0

HuggingFaceAPIToken

Secret details

Encryption key

aws/secretsmanager

Secret name

HuggingFaceAPIToken

Secret ARN

arn:aws:secretsmanager:us-east-1:380106462002:secret:HuggingFaceAPIToken-FJC8ko

Secret description

-

Secret type

-

Actions

Overview

Rotation

Versions

Replication

Tags

Secret value

Info

Retrieve and view the secret value.

Retrieve secret value

Resource permissions - optional

Info

Add or edit a resource policy to access secrets across AWS accounts.

Edit permissions

Sample code

Use these code samples to retrieve the secret in your application.

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

API Gateway

aws

Search

[Option+S]

2

United States (N. Virginia)

Account ID: 3801-0646-2002

susruth-cli

API Gateway

APIs

Routes - hotelreview-api (cbra8z5r6d)

0

API Gateway

APIs

Custom domain names

Domain name access associations

VPC links

API: hotelreview-api...(cbra8z5r6d)

Develop

Routes

Authorization

Integrations

CORS

Reimport

Export

Deploy

Stages

Monitor

Metrics

Logging

Protect

Throttling

Routes

Routes for hotelreview-api

Create

Search

▼ /analytics

▼ /H1

GET

▼ /review

POST

▼ /reviews

▼ /H1

ANY

Choose a route.

Stage: -

Deploy

API Gateway <

APIs

Custom domain names

Domain name access associations

VPC links

API: hotelreview-api...(cbra8z5r6d)

- ▼ **Develop**
- Routes
- Authorization
- Integrations**
- CORS
- Reimport
- Export
- ▼ **Deploy**
- Stages
- ▼ **Monitor**
- Metrics
- Logging
- ▼ **Protect**
- Throttling

Integrations

[Attach integrations to routes](#) | [Manage integrations](#)

Routes for hotelreview-api

▼ /analytics

▼ /H1

GET **AWS Lambda**

▼ /review

POST **AWS Lambda**

▼ /reviews

▼ /H1

ANY **AWS Lambda**

API Gateway

APIs

Custom domain names

Domain name access associations

VPC links

API: hotelreview-api...(cbra8z5r6d)

▼ Develop

Routes

Authorization

Integrations

CORS

Reimport

Export

▼ Deploy

Stages

▼ Monitor

Metrics

Logging

Cross-Origin Resource Sharing

Stage: - Deploy

Configure CORS

CORS allows resources from different domains to be loaded by browsers. If you configure CORS for an API, API Gateway ignores CORS headers returned from your backend integration. See our [CORS documentation](#) for more details.

Access-Control-Allow-Origin

*

Access-Control-Allow-Headers

content-type

Access-Control-Allow-Methods

*

Access-Control-Expose-Headers

3600

Access-Control-Max-Age

0 Seconds

Access-Control-Allow-Credentials

NO

Frontend

Sign In

Create Account

Username

admin

Password

.....

Sign in

Forgot your password?

Sign In

Create Account

Username

Enter your Username

Password

Enter your Password

Confirm Password

Please confirm your Password

Email

Enter your Email

Create Account

Welcome, sruthi

Sign Out

Submit a Review

Your Review

The room was great, but the service was slow...

Rating (1-5)

Submit Review

Recent Reviews

Recent

Top Reviews

Critical Reviews

Rating: 1 / 5

the room is not cleen

Rating: 3 / 5

the room was good but the costs too high

Rating: 2 / 5

the room is good but the food is bad

Rating: 4 / 5

room is fine but the food was a bit cold

Welcome, admin

Sign Out

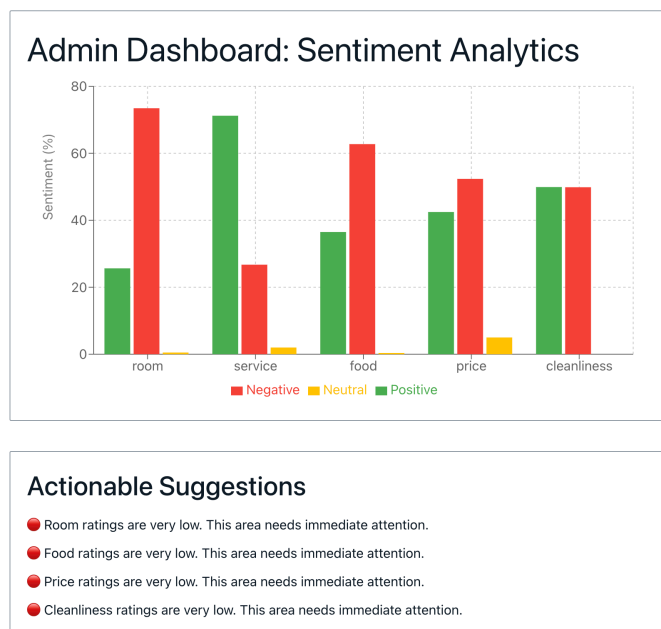
Submit a Review

Your Review

The room was great, but the service was slow...

Rating Please fill out this field.

Submit Review



Front end codes

App.js

```
import { useEffect, useState } from 'react';
import { Authenticator, View, Flex, Loader } from '@aws-amplify/ui-react';
import '@aws-amplify/ui-react/styles.css';
import { ReviewForm } from './ReviewForm';
import { AdminDashboard } from './AdminDashboard';
import { ReviewList } from './ReviewList';
import { fetchAuthSession } from 'aws-amplify/auth';

const AppContent = ({ user, signOut }) => {
  const [isAdmin, setIsAdmin] = useState(false);
  const [isLoading, setIsLoading] = useState(true);
```

```

useEffect(() => {
  const checkUserGroup = async () => {
    try {
      const session = await fetchAuthSession();
      const groups = session.tokens?.idToken.payload["cognito:groups"] || [];
      if (groups.includes('admins')) {
        setIsAdmin(true);
      }
    } catch (error) {
      console.error("Error fetching user session or groups:", error);
    } finally {
      setIsLoading(false);
    }
  };

  if (user) {
    checkUserGroup();
  }
}, [user]);

if (isLoading) {
  return <Loader size="large" margin="20px auto" />;
}

return (
  <View padding="20px">
    <Flex direction="column" alignItems="center">
      <ReviewForm user={user} signOut={signOut} />

      <hr style={{ width: '80%', margin: '40px 0' }} />
      {isAdmin ? (
        <>
          <AdminDashboard />
          <ReviewList />
        </>
      ) : (
        <ReviewList />
      )}
    </Flex>
  </View>
);
};

function App() {
  return (
    <Authenticator>
      {({ signOut, user }) => (
        <AppContent user={user} signOut={signOut} />
      )}
    </Authenticator>
  );
}

```

```

    })
  </Authenticator>
);
}

export default App;

```

AdminDashboard.js

```

import { useEffect, useState } from 'react';
import {
  BarChart,
  Bar,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  Legend,
  ResponsiveContainer
} from 'recharts';
import { Card, Heading, Loader, Text, Flex, Collection } from '@aws-amplify/ui-react';

export const AdminDashboard = () => {
  const [chartData, setChartData] = useState([]);
  const [suggestions, setSuggestions] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchAnalytics = async () => {
      try {
        const analyticsEndpoint = 'https://cbra8z5r6d.execute-api.us-east-1.amazonaws.com/dev/analytics/H1';

        const response = await fetch(analyticsEndpoint);
        if (!response.ok) {
          throw new Error('Failed to fetch analytics. Check API URL and CORS settings.');
        }
        const data = await response.json();

        const formattedData = Object.keys(data.chartData).map(aspect => ({
          name: aspect,
          Positive: data.chartData[aspect].avg_positive * 100,
          Negative: data.chartData[aspect].avg_negative * 100,
          Neutral: data.chartData[aspect].avg_neutral * 100,
        }));
      } catch (error) {
        setError(error);
      }
    };
    fetchAnalytics();
  }, []);

  return (
    <Card>
      <Heading>Admin Dashboard</Heading>
      <Collection>
        <BarChart>
          <ResponsiveContainer>
            <BarChart>
              <XAxis>
                <YAxis>
                <CartesianGrid>
                <Tooltip>
                <Legend>
                <Bar>
              </BarChart>
            </ResponsiveContainer>
          </BarChart>
        </Collection>
      </Card>
    );
  };

```

```

        setChartData(formattedData);
        setSuggestions(data.suggestions);
    } catch (err) {
        setError(err.message);
    } finally {
        setLoading(false);
    }
};
fetchAnalytics();
}, []);

if (loading) return <Loader margin="20px auto" />;
if (error) return <Text color="red">{error}</Text>;

return (
    <Flex direction="column" alignItems="center" width="100%" maxWidth="800px">
        <Card variation="outlined" width="100%" padding="20px">
            <Heading level={2}>Admin Dashboard: Sentiment Analytics</Heading>
            {chartData.length === 0 ? (
                <Text>No analytics data to display. Submit a review first.</Text>
            ) : (
                <ResponsiveContainer width="100%" height={400}>
                    <BarChart
                        data={chartData}
                        margin={{ top: 20, right: 30, left: 20, bottom: 5 }}
                    >
                        <CartesianGrid strokeDasharray="3 3" />
                        <XAxis dataKey="name" />
                        <YAxis label={{ value: 'Sentiment (%)', angle: -90, position:
'insideLeft' }} />
                        <Tooltip />
                        <Legend />
                        <Bar dataKey="Positive" fill="#4CAF50" />
                        <Bar dataKey="Negative" fill="#F44336" />
                        <Bar dataKey="Neutral" fill="#FFC107" />
                    </BarChart>
                </ResponsiveContainer>
            )}
        </Card>
        <Card variation="outlined" width="100%" padding="20px" marginTop="20px">
            <Heading level={3}>Actionable Suggestions</Heading>
            <Collection
                items={suggestions}
                type="list"
                direction="column"
                gap="10px"
                marginTop="15px"
            />
        </Card>
    </Flex>
);

```

```

      >
      {(item, index) => (
        <Text key={index}>{item}</Text>
      )}
    </Collection>
  </Card>
</Flex>
);
};

```

Index.js

```

import React from 'react';
import ReactDOM from 'react-dom/client';
import './index.css';
import App from './App';
import reportWebVitals from './reportWebVitals';

import { Amplify } from 'aws-amplify';
import config from './aws-exports';
Amplify.configure(config);

const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

reportWebVitals();

```

reviewForm.js

```

import { useState } from 'react';
import {
  Button,
  Card,
  Flex,
  Heading,
  Text,
  TextAreaField,
  TextField,
  View
} from '@aws-amplify/ui-react';

export const ReviewForm = ({ user, signOut }) => {
  const [reviewText, setReviewText] = useState('');

```

```

const [rating, setRating] = useState('');
const [submitting, setSubmitting] = useState(false);
const [error, setError] = useState(null);
const [success, setSuccess] = useState(null);

const handleSubmit = async (e) => {
  e.preventDefault();
  setSubmitting(true);
  setError(null);
  setSuccess(null);

  try {

    const apiEndpoint = 'https://cbra8z5r6d.execute-api.us-east-
1.amazonaws.com/dev/review';

    const response = await fetch(apiEndpoint, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json',
      },
      body: JSON.stringify({
        hotelId: 'H1',
        text: reviewText,
        rating: parseInt(rating, 10),
        userId: user.username
      }),
    });

    if (!response.ok) {
      throw new Error('Failed to submit review. Server returned ' +
response.status);
    }

    setSuccess('Thank you! Your review has been submitted.');
    setReviewText('');
    setRating('');
  } catch (err) {
    setError(err.message);
  } finally {
    setSubmitting(false);
  }
};

return (
  <Card variation="outlined" width="100%" maxWidth="800px">
    <View padding="20px">
      <Flex direction="row" justifyContent="space-between" alignItems="center">

```

```

    <Heading level={3}>Welcome, {user.username}</Heading>
    <Button onClick={signOut} variation="link">Sign Out</Button>
  </Flex>

  <Heading level={2} marginTop="20px">Submit a Review</Heading>
  <form onSubmit={handleSubmit}>
    <TextAreaField
      label="Your Review"
      value={reviewText}
      onChange={(e) => setReviewText(e.target.value)}
      placeholder="The room was great, but the service was slow..."
      isRequired
    />
    <TextField
      label="Rating (1-5)"
      type="number"
      min="1"
      max="5"
      value={rating}
      onChange={(e) => setRating(e.target.value)}
      isRequired
    />
    <Button type="submit" variation="primary" isLoading={submitting}
marginTop="20px">
      Submit Review
    </Button>
  </form>
  {success && <Text color="green.80" marginTop="10px">{success}</Text>}
  {error && <Text color="red.80" marginTop="10px">{error}</Text>}
</View>
</Card>
);
};

```

ReviewList.js

```

import { useEffect, useState } from 'react';
import {
  Card,
  Flex,
  Heading,
  Loader,
  Text,
  Collection,
  Badge,
  ToggleButton,
  ToggleButtonGroup

```

```

} from '@aws-amplify/ui-react';

export const ReviewList = () => {
  const [allReviews, setAllReviews] = useState([]);
  const [displayedReviews, setDisplayedReviews] = useState([]);
  const [filterMode, setFilterMode] = useState('recent');
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);

  useEffect(() => {
    const fetchReviews = async () => {
      try {
        const apiEndpoint = 'https://cbr8z5r6d.execute-api.us-east-1.amazonaws.com/dev/reviews/H1';

        const response = await fetch(apiEndpoint);
        if (!response.ok) {
          throw new Error('Failed to fetch reviews. Check API URL and CORS settings.');
        }
        const data = await response.json();
        setAllReviews(data);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };

    fetchReviews();
  }, []);

  useEffect(() => {
    let filtered = [...allReviews];

    if (filterMode === 'top') {
      filtered = allReviews.filter(review => review.rating >= 4);
    } else if (filterMode === 'critical') {
      filtered = allReviews.filter(review => review.rating <= 2);
    }

    setDisplayedReviews(filtered);
  }, [filterMode, allReviews]);

  if (error) return <Text color="red">{error}</Text>;

  return (

```

```

<Card variation="outlined" width="100%" maxWidth="800px" margin="20px auto">
  <Heading level={2} marginBottom="20px">Recent Reviews</Heading>

  <ToggleButtonGroup
    value={filterMode}
    onChange={(value) => setFilterMode(value)}
    isExclusive
    marginBottom="20px"
  >
    <ToggleButton value="recent">Recent</ToggleButton>
    <ToggleButton value="top">Top Reviews</ToggleButton>
    <ToggleButton value="critical">Critical Reviews</ToggleButton>
  </ToggleButtonGroup>

  {loading ? (
    <Loader margin="20px auto" />
  ) : displayedReviews.length === 0 ? (

    <Text>No reviews found for this filter.</Text>
  ) : (
    <Collection
      items={displayedReviews}
      type="list"
      direction="column"
      gap="20px"
      height="500px"
      overflow="auto"
    >
      {(item, index) => (
        <Card key={index} variation="outlined">
          <Flex direction="column">
            <Heading level={5}>Rating: {item.rating} / 5</Heading>
            <Text>{item.text}</Text>

            {item.badges && item.badges.length > 0 && (
              <Flex direction="row" gap="10px" marginTop="10px">
                {item.badges.map((badge, badgeIndex) => (
                  <Badge
                    key={badgeIndex}
                    variation={badge === "Top Reviewer" ? "success" : "info"}
                  >
                    {badge}
                  </Badge>
                ))}
              </Flex>
            )}
          </Flex>
        </Card>
      )}
    </Collection>
  )}
</Flex>

```

```
        </Card>
      )}
    </Collection>
  )}
</Card>
);
};
```